

Relazione

Progetto di integrazione di Ingegneria del Software

1. Introduzione

Lo scopo di questo progetto è stato quello di riprogettare, ricodificare e ritestare alcune componenti del progetto *Sistema di Gestione di un Magazzino Distribuito*, sviluppato dal gruppo *Code Alchemists* durante il corso di Ingegneria del Software nell'A.A. 2025/2026.

L'attività si è concentrata principalmente sulla revisione dell'architettura dei microservizi relativi alla gestione dei singoli magazzini e all'aggregazione dei dati provenienti da essi. È stata effettuata una riprogettazione del dominio e contestualmente delle classi, sono state implementate da zero le componenti coinvolte e sono state svolte e automatizzate diverse suite di test di unità, di integrazione e di sistema.

L'obiettivo è stato quindi quello di individuare le principali criticità architetturali del sistema originale e di correggerle. Ciò è stato fatto con una nuova modellazione dei bounded context, con una maggiore separazione delle responsabilità e un'implementazione più curata dei design pattern.

2. Analisi del sistema originale

2.1 Architettura originale

Il sistema originale era composto dalle seguenti componenti principali:

- Centralized
 - *authentication*: gestione dell'autenticazione e dell'autorizzazione degli utenti;
 - *CentralSystem*: coordinamento centrale e orchestrazione delle operazioni;
 - *routing*: gestione del routing degli ordini e delle spedizioni;
 - *state_aggregate*: aggregazione e monitoraggio dello stato globale.
- Cloud
 - *orders-aggregated*: aggregazione e sincronizzazione degli ordini tra i magazzini;
 - *inventory-aggregated*: aggregazione e sincronizzazione dell'inventario.
- Warehouse
 - *orders*: gestione degli ordini locali;
 - *inventory*: gestione dell'inventario locale;
 - *state*: gestione dello stato del magazzino.
- Monitoring
 - *Grafana*: visualizzazione delle metriche;
 - *Prometheus*: raccolta e scraping delle metriche.

2.2 Criticità individuate

L'analisi dell'architettura originale ha evidenziato una modellazione non ottimale di alcuni bounded context. In particolare, le responsabilità relative alla gestione di un singolo magazzino erano erroneamente distribuite tra i microservizi *orders*, *inventory* e *state*.

Sebbene tali componenti rappresentassero responsabilità logicamente distinte, vi era una forte correlazione tra esse, in quanto dovevano collaborare frequentemente per portare a termine le operazioni di dominio del magazzino. Questa suddivisione introduceva quindi un accoppiamento elevato tra le componenti appartenenti allo stesso contesto applicativo.

Una criticità analoga era presente nella componente cloud, dove i dati relativi agli ordini e all'inventario dei magazzini erano distribuiti su due distinti servizi di aggregazione, nonostante entrambi fossero utilizzati per fornire una visione complessiva dei magazzini.

L'analisi ha inoltre evidenziato un'errata suddivisione delle responsabilità tra livello di dominio e livello applicativo. In particolare, alcune operazioni di modifica dello stato degli aggregati erano state collocate all'interno di classi di servizio invece di essere incapsulate nell'aggregato responsabile del mantenimento delle relative invarianti.

2.3 Obiettivi della riprogettazione

A partire dalle criticità individuate, la riprogettazione si è quindi basata sui seguenti obiettivi:

1. definire bounded context maggiormente coerenti con le responsabilità del dominio;
2. ridurre l'accoppiamento tra i microservizi relativi allo stesso magazzino, mantenendo alta la coesione;
3. correggere le responsabilità di dominio restituendole agli aggregati;
4. mantenere separati i flussi di modifica e di lettura dei dati come da pattern CQRS;
5. rendere il sistema maggiormente estensibile e testabile.

3. Progettazione

3.1 Warehouse

3.1.1 Identificazione del bounded context e modellazione del dominio

In fase di progettazione, la gestione di un singolo magazzino è stata identificata come un sottodominio del sistema e modellata attraverso un bounded context corrispondente al microservizio warehouse.

Un magazzino deve gestire le informazioni relative all'inventario dei prodotti contenuti, agli ordini inseriti e allo stato del magazzino stesso. La definizione degli aggregati all'interno di questo bounded context ha quindi richiesto particolare attenzione.

Una delle prime scelte progettuali ha riguardato la scelta della granularità degli aggregati. Una regola fondamentale adottata nella modellazione è che le operazioni sul contenuto di un aggregato debbano essere effettuate attraverso la relativa aggregate root. Una possibile soluzione avrebbe quindi previsto un aggregato Warehouse contenente i prodotti e gli ordini del magazzino. Tuttavia, una simile soluzione avrebbe imposto di eseguire ogni modifica di un prodotto o di un ordine passando dalla root Warehouse, aumentando le dimensioni e le responsabilità dell'aggregato.

Considerando inoltre il principio secondo cui un aggregato dovrebbe essere mantenuto quanto più piccolo possibile, si è scelto di modellare Product e Order come aggregate root distinti. Questa scelta consente di modificare i prodotti e gli ordini indipendentemente senza introdurre un unico aggregato di dimensioni eccessive.

Lo stesso ragionamento ha portato all'introduzione di un aggregato Reservation, necessario per rappresentare esplicitamente il ciclo di vita delle prenotazioni associate agli ordini.

Invece, l'aggregato responsabile della gestione dello stato del magazzino e tutte le altre componenti del relativo microservizio preesistente sono state mantenute *as-is*, incapsulandole però come componente del nuovo microservizio *warehouse* alla pari degli altri moduli sopra descritti.

La modellazione risultante ha permesso quindi di mantenere separati i principali elementi del dominio:

- Product, responsabile dello stato e delle operazioni sul singolo prodotto;
- Order, responsabile dello stato e delle informazioni degli ordini;
- Reservation, responsabile della prenotazione dei prodotti associati a un ordine.
- State, responsabile della gestione dello stato del magazzino stesso.

Questa struttura evita di concentrare le responsabilità in un unico aggregato Warehouse e rende più esplicite le invarianti e i cicli di vita dei singoli elementi del dominio.

3.1.2 Order Saga

Un secondo problema progettuale rilevante ha riguardato la gestione del ciclo di vita degli ordini.

Le operazioni necessarie alla gestione di un ordine coinvolgono più aggregati e, in alcuni casi, più microservizi. Non è quindi possibile coordinare l'intero processo in una singola transazione ACID. È stato pertanto necessario adottare un meccanismo di coordinamento tramite saga, accettando la possibilità di stati intermedi e prevedendo operazioni compensative in caso di fallimento.

La progettazione della saga è stata effettuata iterativamente mediante una macchina a stati finiti. Questo approccio ha permesso di rappresentare esplicitamente gli stati dell'ordine e le transizioni causate dagli eventi prodotti durante il processo.

Lo studio della saga è iniziato affrontando il problema della prenotazione dei prodotti. La prima fase del flusso consiste infatti nella creazione dell'ordine e nella successiva

pre-prenotazione degli elementi disponibili. In seguito i prodotti vengono caricati dal repository, modificati tramite le operazioni dell'aggregato Product e salvati nuovamente. Il completamento di ogni operazione genera eventi di dominio relativi ai prodotti prenotati.

Tali eventi vengono successivamente utilizzati per aggiornare l'aggregato Reservation. La reservation rappresenta quindi lo stato delle quantità effettivamente prenotate per l'ordine e costituisce un elemento necessario per poter validare successivamente la corrispondenza tra richiesta dell'ordine e quantità disponibili.

La validazione della reservation costituisce quindi il punto decisionale principale della saga. Se le quantità prenotate soddisfano l'ordine, l'elaborazione procede verso la spedizione. Nel caso contrario, viene inviata una richiesta di rifornimento al sistema centrale. Al completamento del rifornimento, la saga riprende il flusso di elaborazione dell'ordine a partire dalla fase di (ri)prenotazione dei prodotti mancanti.

Nel caso in cui si verifichi un errore imprevisto durante la spedizione, viene avviata una procedura compensativa di cancellazione dell'ordine. In questa fase vengono caricati i dati inerenti alla relativa prenotazione per poter rilasciare le corrette quantità di prodotti.

Analogamente, in caso di successo della spedizione, il completamento dell'operazione genera un evento che permette alla saga di procedere verso la fase di consegna.

La consegna di un ordine è simulata a seconda del tipo di ordine: un ordine di vendita può considerarsi consegnato una volta spedito. Ordini di trasferimento attendono una notifica di avvenuta ricezione della merce da parte dell'altro magazzino. Ordini di rifornimento si comportano come ordini di trasferimento, ma generano inoltre un evento che permette alla saga dell'ordine in attesa di quella merce di riprendere.

La saga funziona tramite un meccanismo evento-comando. Ogni evento di dominio o applicativo di interesse relativo agli ordini è associato ad un comando che permette alla saga di proseguire mediante transizioni predeterminate.

3.1.3 Design pattern

Durante la fase di studio sono stati approfonditi i pattern CQRS, Command, Saga ed Event Sourcing e ne è stata valutata l'applicabilità al progetto.

In particolare, l'Event Sourcing è stato analizzato come possibile strategia di persistenza degli aggregati. È stato tuttavia scelto di non adottarlo nel progetto, poiché nel contesto considerato il rapporto tra benefici ottenibili e complessità introdotta non risultava favorevole.

È stata invece adottata una separazione tra flussi di comando e di query mediante CQRS. La scelta è risultata coerente con la natura event-driven del sistema, con la presenza della saga e con la necessità di mantenere separati il modello utilizzato per modificare il dominio e quello utilizzato per effettuare le query.

Sono stati pertanto introdotti read model dedicati ai prodotti e agli ordini, aggiornati attraverso proiezioni degli eventi di dominio. Il modello di lettura della reservation non è stato

implementato, non essendo necessario per i casi d'uso previsti, ma la struttura del sistema ne permette l'eventuale introduzione futura.

3.1.4 Processo di progettazione

La progettazione non è stata svolta in un'unica iterazione, ma attraverso una serie di approfondimenti progressivi.

In una prima fase è stato studiato il Domain-Driven Design, con particolare attenzione ai concetti di bounded context, aggregate root ed eventi di dominio. Parallelamente sono stati analizzati i possibili aggregati del sottodominio relativo al magazzino e sono stati definiti i primi eventi e relativi trigger.

Successivamente è stato approfondito il concetto di saga ed è stata valutata la modalità di coordinamento più idonea, tra orchestrazione e coreografia. Come detto in precedenza, la saga dell'ordine è stata quindi progettata in più iterazioni come macchina a stati finiti, affrontando progressivamente le problematiche relative alla prenotazione dei prodotti, al rifornimento e alle operazioni compensative.

In una fase successiva, gli stati e le transizioni della macchina a stati sono stati tradotti negli elementi concreti dell'implementazione, ossia command, command handler, eventi applicativi, e sono state svolte prove di codifica per verificare la fattibilità delle soluzioni progettate.

Lo studio è proseguito con la definizione della struttura del microservizio e con la realizzazione dei diagrammi delle classi a partire dal modello di dominio, e del diagramma di flusso per la saga degli ordini.

Infine, sono state effettuate prove di codifica sulla creazione e pubblicazione degli eventi, verificando concretamente il meccanismo event-driven alla base dell'implementazione.

3.2. Warehouse Aggregator

I precedenti servizi *orders-aggregated* e *inventory-aggregated* sono stati accorpati nel nuovo microservizio *warehouse-aggregator*.

La responsabilità del servizio è mantenere una vista complessiva dei dati relativi ai magazzini, rendendoli disponibili esclusivamente tramite operazioni di lettura. Il servizio *state_aggregate*, che rappresenta invece un bounded context distinto relativo alla gestione dello stato globale, è stato mantenuto separato.

Il nuovo servizio warehouse-aggregator è stato riprogettato sfruttando i principi di CQRS applicati a servizi query-only. Esso non contiene quindi logica di dominio relativa alla gestione degli ordini o dell'inventario. Riceve gli aggiornamenti prodotti dai magazzini attraverso il meccanismo di comunicazione event-driven e aggiorna di conseguenza i propri read model.

La funzione del microservizio è quindi principalmente quella di proiettare gli aggiornamenti ricevuti in un repository di sola lettura. Non vengono eseguite operazioni di business sul dominio del magazzino; l'input ricevuto viene validato tramite DTO e trasformato nella relativa proiezione.

Questo approccio consente di separare nettamente il processo di aggiornamento dei dati dal loro consumo tramite query, implementando quindi una variante del meccanismo CQRS come illustrata nel libro *Microservices Pattern* di C. Richardson (Cap. 7, paragrafo CQRS AND QUERY-ONLY SERVICES).

Vi è una piena separazione delle due componenti accorpate, risultando quindi essere moduli indipendenti del servizio.

4. Codifica

La codifica è stata preceduta da uno studio di sistemi utilizzando tecnologie e architetture simili.

È stato utilizzato come riferimento il progetto open source [nestjs-rest-cqrs-example](#), principalmente per valutare possibili modalità di organizzazione del codice relative a CQRS e NestJS.

La codifica è stata condotta seguendo un approccio inside-out, coerente con la struttura definita durante la progettazione. Si è partiti dalle componenti centrali del dominio per procedere progressivamente verso i livelli applicativo e infrastrutturale.

A partire da una fork del progetto MVP originale, sono stati rimossi i servizi interessati dalla riprogettazione e sono state implementate ex-novo le componenti previste dal nuovo modello.

Il processo di codifica è stato suddiviso in parti e tracciato mediante checklist, data l'elevata complessità e numerosità di elementi del sistema, in modo da mantenere una visione di insieme complessiva.

In particolare, la codifica del servizio *warehouse* ha seguito queste fasi:

- definizione dei value object e delle enumerazioni;
- implementazione degli aggregati Product, Reservation e Order;
- definizione delle interfacce dei repository;
- implementazione degli eventi di dominio;
- definizione degli eventi e delle interfacce utilizzate per la comunicazione tra microservizi;
- definizione dei read model e dei relativi DTO;
- implementazione degli use case;
- implementazione di command e query;
- implementazione dei service;
- realizzazione degli schema MongoDB e delle implementazioni dei repository;
- implementazione di query handler, event handler e command handler;

- implementazione della Order Saga;
- implementazione delle porte in ingresso e in uscita;
- implementazione dei controller HTTP.
- implementazione su branch separato delle componenti preesistenti del microservizio state, modificandone solo la struttura delle cartelle, con conseguente merge.

Il servizio *warehouse-aggregator* è stato implementato in maniera analoga.

I componenti sono stati integrati mediante i moduli e i meccanismi di Dependency Injection di NestJS, utilizzando i relativi decoratori per collegare le diverse componenti del sistema.

L'approccio adottato ha permesso di procedere in maniera incrementale, data l'elevata complessità del sistema, permettendo di mantenere una visione di insieme delle diverse parti dell'implementazione.

5. Testing

5.1 Strategia di testing

La validazione del sistema riprogettato è stata effettuata attraverso tre differenti livelli di test:

- test di unità, finalizzati alla verifica del comportamento di singole componenti isolate;
- test di integrazione, finalizzati alla verifica della collaborazione tra componenti e, in particolare, dell'interazione con l'infrastruttura reale;
- test di sistema, finalizzati alla verifica dei comportamenti distribuiti su più microservizi e dei flussi operativi.

La separazione delle responsabilità introdotta dalla nuova architettura ha facilitato in particolare la realizzazione dei test unitari, grazie alla possibilità di sostituire le dipendenze esterne con mock.

5.2 Coverage

Le percentuali di copertura ottenute localmente mediante Jest sono riportate nella tabella seguente:

Servizio	% Stmts	% Branch	% Funcs	% Lines
Warehouse	93.66	77.67	88.28	93.96
Warehouse-aggregator	95.62	76.11	89.85	94.95
Central System	90.42	73.58	86.22	90.49
Authentication	79.76	65.93	84.09	80.08
State-aggregate	91.84	75	85.24	91.5
Routing	97.37	82.88	93.33	97.84

I valori riportati da Codecov, visibili nella descrizione del repository su Github, risultano leggermente differenti rispetto a quelli ottenuti localmente tramite Jest. La differenza è dovuta ad una diversa modalità di calcolo delle copertura del codice da parte di Codecov.

La coverage non viene considerata, di per sé, una misura sufficiente della qualità del software, ma rappresenta un indicatore quantitativo dell'estensione della suite di test. Infatti, alcuni comportamenti del sistema complessivo non sono stati testati.

Nel complesso, la coverage del sistema riportata dalla CI tramite Codecov è passata dall'83% del sistema originale all'88% nel sistema riprogettato.

5.3 Test di unità e integrazione

Per i microservizi warehouse e warehouse-aggregator sono stati realizzati i seguenti test:

	Warehouse	Warehouse-aggregator
Test unitari	224	30
Test di integrazione	31	14
Totale	255	44
Superati	255	44

I test unitari hanno fatto ampio uso di mock per migliorare la testabilità della componente oggetto di test.

I test di integrazione hanno permesso invece di verificare il comportamento delle componenti in presenza delle infrastrutture reali utilizzate dal sistema, in particolare MongoDB e, negli scenari che lo richiedono, NATS.

Tra gli integration test sono presenti test relativi alla persistenza degli aggregati e dei read model. Questi test hanno avuto un ruolo importante nell'individuazione di problematiche non rilevabili tramite i soli test unitari.

5.4 System test

Sono stati inoltre realizzati alcuni test di sistema per verificare il comportamento tra i vari microservizi.

Gli scenari testati comprendono la sincronizzazione delle informazioni tra un warehouse e il warehouse-aggregator, verificando che gli aggiornamenti prodotti dal magazzino vengano correttamente propagati attraverso il sistema di messaggistica e successivamente proiettati nel database dell'aggregatore.

In particolare, sono stati verificati:

- la sincronizzazione della creazione di un prodotto da un magazzino al warehouse-aggregator;
- la sincronizzazione della creazione di un ordine da un magazzino al warehouse-aggregator.

Inoltre, sono stati testati alcuni scenari relativi agli ordini, tra cui:

- il completamento di un ordine di vendita a partire dalla sua creazione;
- il completamento di un ordine di trasferimento a partire dalla sua creazione.

La presenza di test di sistema ha permesso quindi di verificare non soltanto la correttezza delle singole implementazioni, ma anche la corretta interazione tra i servizi e le infrastrutture di comunicazione e persistenza.

5.5 Continuous Integration

L'esecuzione automatizzata dei test è stata integrata nella pipeline di GitHub Actions.

La CI è stata organizzata in job distinti per i diversi microservizi, consentendo l'esecuzione parallela delle rispettive suite di test, riducendo così i tempi complessivi della pipeline.

Per i test che richiedono infrastruttura esterna vengono avviati container dedicati tramite una configurazione Docker Compose specifica per l'ambiente di test. In questo modo è possibile utilizzare istanze isolate di MongoDB e NATS durante l'esecuzione automatica.

Sono inoltre stati utilizzati:

- *npm ci* per garantire installazioni riproducibili a partire dai rispettivi lockfile;
- caching delle dipendenze npm per ridurre i tempi di installazione;
- *Docker Compose* per l'avvio dell'infrastruttura necessaria ai test di integrazione e di sistema;
- *Codecov* per la raccolta e il monitoraggio della coverage.

La pipeline comprende infine una parte dedicata ai system test, nella quale vengono avviati i microservizi necessari ed eseguiti in maniera automatica i test.

È stato quindi aggiunto un badge relativo alla CI nel Readme del repository, oltre a quelli di Codecov, in modo da avere un indicatore chiaro dello stato della pipeline e dell'eventuale presenza di regressioni introdotte da modifiche al codice.

6. Considerazioni finali

La riprogettazione ha consentito di intervenire sulle principali criticità individuate nell'architettura originale, mantenendo al contempo le funzionalità fondamentali del sistema.

La modifica principale ha riguardato l'identificazione del magazzino come bounded context, con il conseguente accorpamento delle precedenti componenti orders, inventory e state nel nuovo servizio warehouse. Analogamente, le componenti di aggregazione degli ordini e

dell'inventario sono state riunite nel warehouse-aggregator, progettato come microservizio query-only.

Nel nuovo warehouse è stato effettuato un miglioramento nella modellazione degli aggregati, è stata modellata la gestione delle prenotazioni tramite un aggregato dedicato, è stata riprogettata la Order Saga per la coordinazione delle operazioni distribuite, ed è stato implementato il pattern CQRS separando il modello di scrittura quello di lettura.

La nuova organizzazione ha inoltre migliorato la testabilità del sistema. La presenza di una suite composta da test unitari, di integrazione e di sistema ha permesso di verificare il comportamento delle componenti su diversi livelli di isolamento e di individuare problemi che non sarebbero emersi tramite i soli test unitari.

Sebbene il testing del sistema non sia esaustivo, è stata predisposta l'infrastruttura necessaria per poter estendere la validazione ai restanti flussi del sistema.

Nel complesso, il progetto ha portato a una maggiore coesione delle responsabilità all'interno dei microservizi riprogettati, a una migliore separazione tra dominio, applicazione e infrastruttura e a una pipeline di testing automatizzata in grado di verificare sia le singole componenti sia le principali interazioni tra microservizi.

La nuova architettura garantisce inoltre una buona estensibilità del sistema. Tra i possibili sviluppi futuri vi sono l'estensione dei read model e query alle Reservation, l'eventuale introduzione di meccanismi di event storing o event sourcing e l'estensione della copertura dei test di sistema ad altri flussi distribuiti.